CLASS : B. E. (E &TC) **SUBJECT : ESD**

EXPT. NO. : 08 **DATE :**

Aim:

Learn how to Configure Free RTOS Using CubeMX.

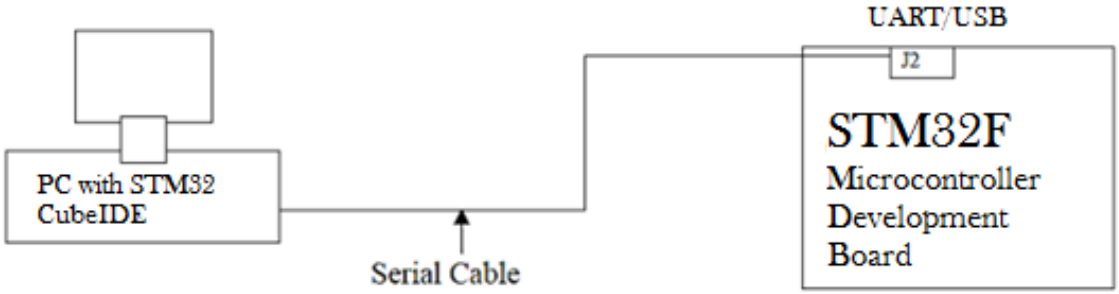
Objectives:

Create project and write code in FreeRTOS of STM32F microcontroller.

Requirements:

STM32FXX Microcontroller Development Board, Personal Computer, STM32CubeIDE Software, Connecting wires, Power Supply, etc.

Block Diagram:



Theory:

RTOS stands for Real Time Operating System. And as the name suggests, it is capable of doing tasks, as an operating system does. The main purpose of an OS is to have the functionality, where we can use multiple tasks at the same time. Which obviously isn't possible with bare metal.

This tutorial is the first in the series of many, and will cover the following:-

1.) Setting up Free RTOS using CubeMX

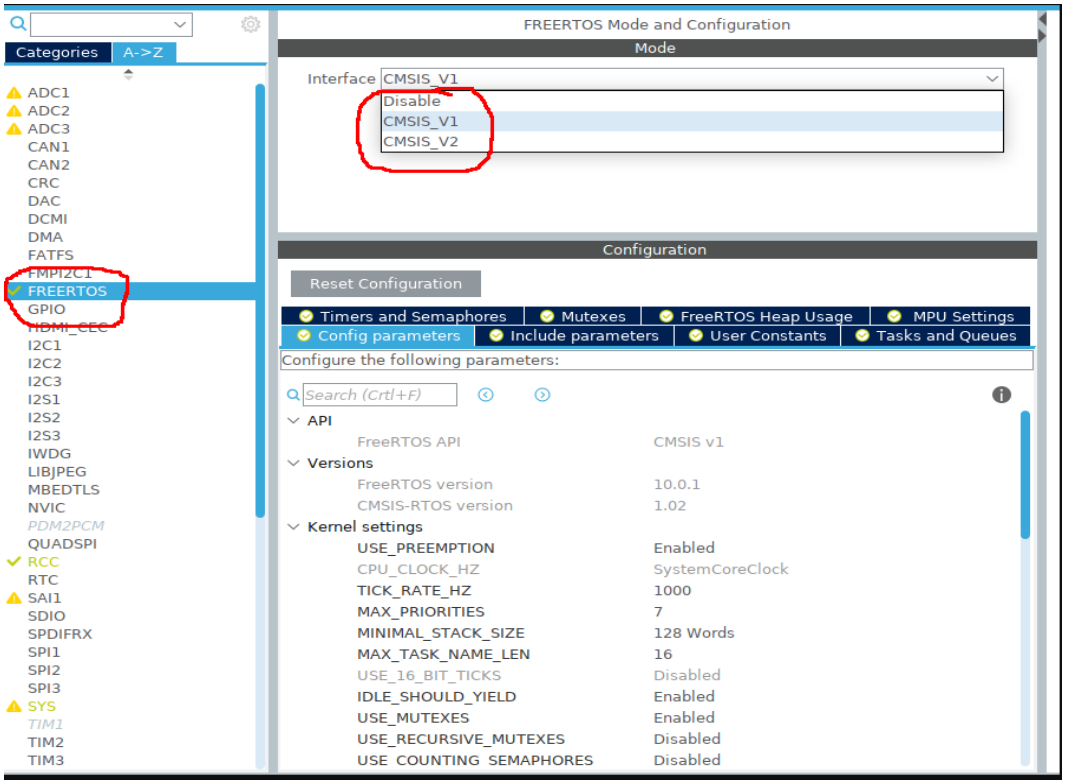
2.) Benefit of using a RTOS.

3.) Creating tasks with or without CubeMX

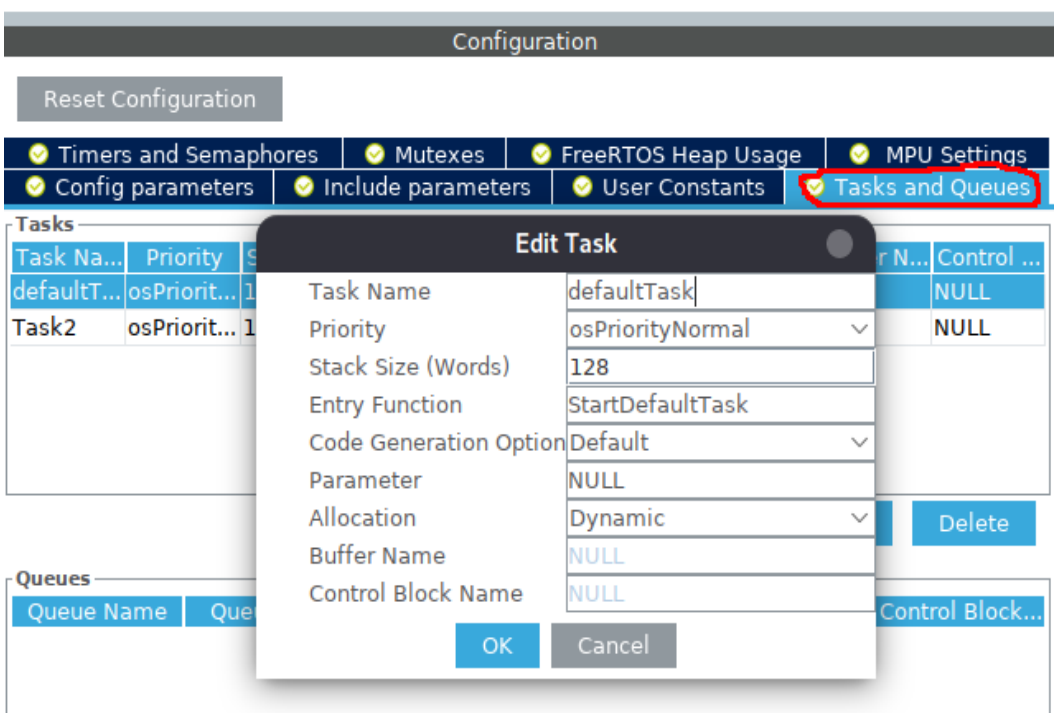
4.) Using priorities to sort out some common problems

CubeMX Setup:

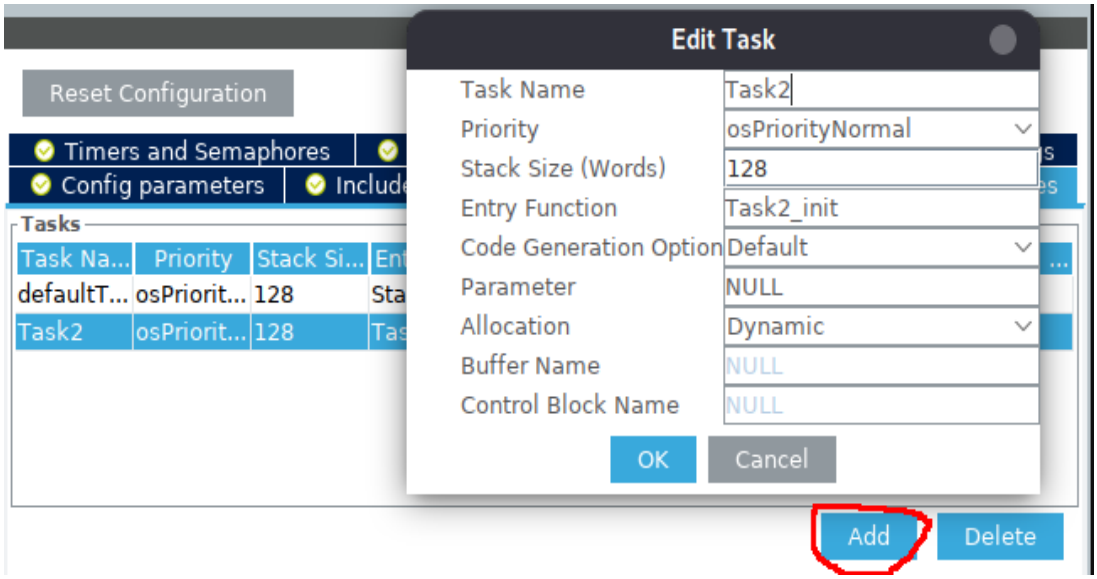
- After selecting the controller, the CubeMX will open the default page for you. Now select the **FREERTOS** and follow the screenshot below



- Next, go to the 'tasks and queues' tab and here you will see a default task, already created for you. Double click it and you can see the following information.

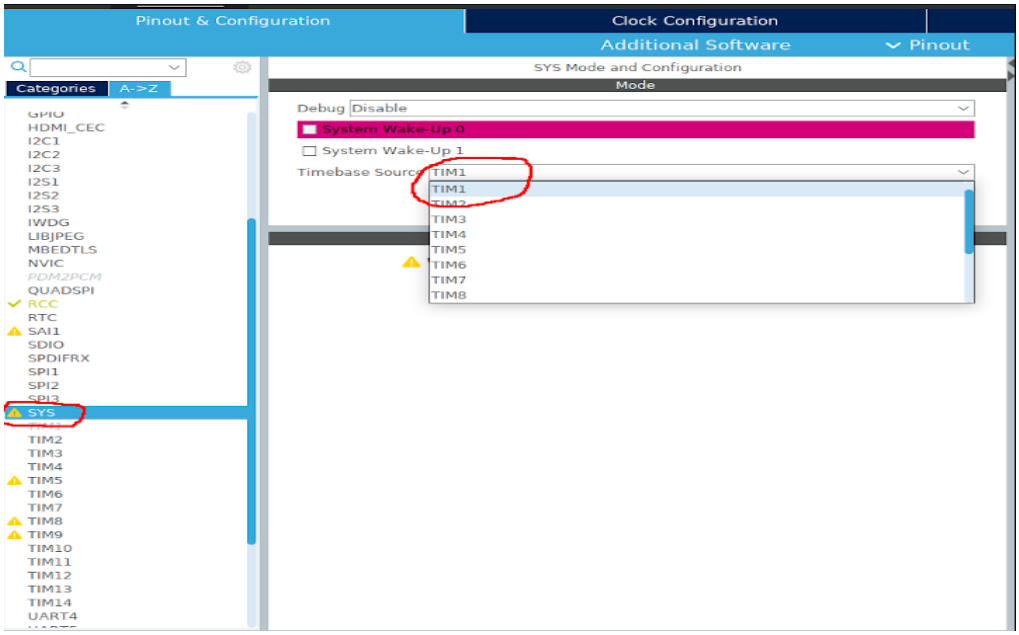


- Now, we will create one task here, and below are the properties of that task.



- So we are calling it task 2, with normal priority, and the entry function is Task2_init. You will get a better idea about these, once we write the program.

- Also one important thing about using RTOS is that, we can't use systick as the time base. So go to sys, and choose some other timebase as shown below



- Other than these, we are using UART 2 for transmitting data, and pins PA0, and PA1 as output. After the code is generated, open the main.c file we have to define the task1 and task2.

Defining the tasks:

- we want to toggle 2 pins, and each at some respective delays. So basically, we want both the pins to toggle at the same time.

```
void StartDefaultTask(void const * argument)
{
  /* USER CODE BEGIN 5 */
  /* Infinite loop */
  for(;;)
  {
     HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_0);
     osDelay(1);
  }
  /* USER CODE END 5 */
}

/* USER CODE BEGIN Header_Task2_init */
/**
* @brief Function implementing the Task2 thread.
* @param argument: Not used
* @retval None
*/
/* USER CODE END Header_Task2_init */
void Task2_init(void const * argument)
{
  /* USER CODE BEGIN Task2_init */
  /* Infinite loop */
  for(;;)
  {
          HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_1);
          osDelay(1);
  }
  /* USER CODE END Task2_init */
}
```

Building the Project:

Follow the debug and execution procedure mentioned in the previous experiment.

Conclusion: _____

_____

_____